MICRO PROJECT 14

MODEL AND PREDICT WEBSITE TRAFFIC OVER TIME

Product: TrafficIQ

Institution: [Insert verified institution name]

Department: [Insert department]

Student name: [Insert name]

Register number: [Insert register number]

Academic year: [Insert academic year]

Certificate

[Insert the institution-approved certificate wording, guide name, department seal and authorised signatures. Do not use fabricated signatures.]

Declaration

[Insert the student’s own declaration and signature.]

Acknowledgement

[Insert acknowledgement in the student’s own words.]

Abstract

TrafficIQ is a data-first application designed to analyse historical website traffic and estimate future visitor volume. It combines a React and TypeScript frontend with a Python FastAPI backend and a relational database. The implemented foundation includes site management, CSV validation and ingestion, descriptive analytics, polynomial calculus utilities, baseline forecasting, chronological model evaluation, robust anomaly detection, visitor-volume capacity calculations and report export endpoints. The system deliberately avoids synthetic traffic records and fabricated performance metrics. Results are displayed only when genuine observations are supplied. This report is an editable project draft; institution-specific details, screenshots and experiments with an authentic dataset must be added before final submission.

1. Introduction

Website traffic changes over time due to audience behaviour, content updates, campaigns, seasonality and external events. Analysing recorded traffic can reveal patterns and help estimate future visitor demand. A trustworthy forecasting system must preserve data provenance, validate its methods and evaluate predictions against observations that were not used for fitting.

2. Problem Statement

A user needs an application that imports genuine website traffic observations, analyses temporal patterns and applies mathematical models to estimate future traffic without presenting invented data as evidence.

3. Objectives

Import and preserve genuine observations and source information.

Calculate descriptive traffic measures from stored records.

Demonstrate derivatives, curvature, critical points and numerical integration on fitted traffic curves.

Compare baseline forecasting methods using chronological evaluation.

Identify unusual observations and estimate visitor-volume headroom.

Export calculated findings for review.

4. Existing System and Limitations

Basic analytics dashboards may show historical counts but may not explain calculus-based modelling, forecast validation and capacity assumptions in one workflow. Any analysis that uses unverified or fabricated values can lead to misleading conclusions.

5. Proposed System

TrafficIQ provides a browser-based analytics workspace, a FastAPI service layer and a relational data store. The initial implementation supports CSV-based observations, calculated summaries, calculus analysis, baseline forecasting, anomaly detection, capacity calculations and downloadable summaries. Google Analytics 4 integration and production-grade first-party tracking are not yet implemented.

6. Literature and Background Study

The project uses official technical documentation for the frameworks and numerical libraries listed in the reference register. Academic literature should be added only after the exact publication has been consulted and verified.

7. Mathematical Foundation

Let V(t) denote a fitted visitor-traffic curve. The first derivative V'(t) estimates the local rate of change. The second derivative V''(t) describes curvature. Interior candidate critical points satisfy V'(t)=0 and are classified by checking derivative signs around the point. An inflection candidate is accepted only when the second derivative changes sign. The trapezoidal rule approximates the area under the observation curve; this quantity is distinct from the discrete sum of visitor counts. Polynomial regression is bounded in degree and should not be extrapolated outside its evidence range without validation. Logistic growth is documented as dV/dt=rV(1-V/K), but fitting is not yet implemented in this build.

8. System Requirements

Frontend: React, TypeScript, Vite, React Router and Recharts. Backend: Python, FastAPI, Pydantic and SQLAlchemy. Database: PostgreSQL target, with SQLite available only as a local development fallback. Analysis: NumPy, Pandas, SciPy and scikit-learn. Testing: Pytest and planned Vitest/browser tests.

9. System Architecture

The frontend calls versioned FastAPI endpoints. The backend validates requests, reads and writes relational entities, and delegates calculations to independent service modules. The initial schema includes sites, datasets, traffic observations, tracking events, forecasts, evaluations, anomalies and capacity configurations.

10. Database Design

The schema uses primary keys, foreign keys, indexes and uniqueness constraints. Dataset provenance is stored with each import. Observations reference a site and dataset. The current owner identifier is a development placeholder, not a secure authentication principal; authenticated ownership enforcement is required before public deployment.

11. Methodology

CSV timestamps are parsed and normalised to UTC. Required columns are timestamp and visitors; sessions and page_views are optional. Missing values, invalid timestamps, negative or non-integer counts and duplicate timestamps are rejected. Forecast evaluation uses chronological holdout splitting rather than random shuffling. MAE, RMSE and sMAPE are calculated from held-out predictions.

12. Implementation

The initial build contains a responsive React dashboard with light/dark theme support, a data-management upload interface, FastAPI routes, SQLAlchemy models, an Alembic migration and separate analysis services. The dashboard remains empty until a user imports or collects genuine observations.

13. Testing and Results

Backend syntax compilation completed. Eight backend service tests passed in the available environment. A local SQLite migration and API smoke test also completed using an isolated temporary database. The smoke test used disposable test-only observations and is not evidence of real website performance. No genuine user dataset has been evaluated in this build. Frontend dependency installation was blocked by package-registry access, so frontend build and browser-test results are pending. PostgreSQL integration testing is also pending.

14. Discussion

The initial foundation demonstrates how descriptive analytics, calculus utilities, forecasting baselines, anomaly detection and capacity calculations can be separated from API routing. The project’s current limitations must be considered when interpreting the result: authentication is not production-ready, GA4 is not connected, tracking events are not yet aggregated into traffic observations, logistic fitting and prediction intervals are not implemented, and several module visualisations remain incomplete.

15. Limitations and Future Enhancements

Implement secure authentication and ownership enforcement. Complete the privacy-conscious first-party tracking pipeline and event aggregation. Implement official GA4 authentication and synchronisation. Add time-granularity aggregation and heatmaps. Fit and validate logistic growth where appropriate. Add supported uncertainty intervals, richer model comparisons and full report generation. Complete frontend tests, PostgreSQL validation and deployment hardening.

16. Conclusion

TrafficIQ establishes a real-data-oriented foundation for website traffic analysis and mathematical forecasting. Its value depends on genuine observations, transparent assumptions, chronological validation and honest reporting of unfinished capabilities.

References

FastAPI documentation: https://fastapi.tiangolo.com/

React documentation: https://react.dev/

TypeScript documentation: https://www.typescriptlang.org/docs/

Vite documentation: https://vite.dev/guide/

SQLAlchemy documentation: https://docs.sqlalchemy.org/

Alembic documentation: https://alembic.sqlalchemy.org/

PostgreSQL documentation: https://www.postgresql.org/docs/

NumPy documentation: https://numpy.org/doc/

SciPy documentation: https://docs.scipy.org/doc/scipy/

pandas documentation: https://pandas.pydata.org/docs/

scikit-learn documentation: https://scikit-learn.org/stable/

Google Analytics Data API documentation: https://developers.google.com/analytics/devguides/reporting/data/v1

Appendix

Insert actual application screenshots after launching the completed frontend. Insert verified test logs and results from a genuine dataset. Add installation and configuration details relevant to the final submission environment.